

图像边缘检测与分类实验报告

图像边缘检测任务

实验原理

在本次实验中，使用了Sobel算法进行边缘检测。

图像的相邻区域之间，像素值是不连续的。因此，在图像的边缘处，像素值的梯度最大。

Sobel算法就是基于这一原理。在进行图像分割时，首先计算每个像素沿x、y方向的梯度，然后利用公式：

$$G = \sqrt{G_x^2 + G_y^2}$$

来计算每个像素点的边缘强度。最后，根据边缘强度进行二值化，得到边界点和非边界点，进而实现图像分割。

实验过程

首先，引入opencv包，并准备好原始图像。原始图像如下图：



第二步，读入图像。为了方便后续处理，图像需要以灰度模式读入。

```
graph = cv2.imread('WaTaShi.jpg', cv2.IMREAD_GRAYSCALE)
graph1 = cv2.imread('WaTaShi.jpg')
# 求梯度
```

第三步，使用sobel算法进行图像分割。

```
# 求梯度
grad_x = cv2.Sobel(graph, cv2.CV_64F, dx: 1, dy: 0, ksize=3)
grad_y = cv2.Sobel(graph, cv2.CV_64F, dx: 0, dy: 1, ksize=3)
# 求边缘强度
combination = cv2.magnitude(grad_x, grad_y)
# 转化为unit8
grad_x = cv2.convertScaleAbs(grad_x)
grad_y = cv2.convertScaleAbs(grad_y)
combination = cv2.convertScaleAbs(combination)
# 图像分割
value, binary_edge = cv2.threshold(combination, thresh: 50, maxval: 255, cv2.THRESH_BINARY)
```

最后，调整图像大小，在屏幕上显示。如果按照默认大小，笔记本电脑屏幕上将无法显示完整图像。

实验结果



图像分类任务

实验准备

安装 `tensorflow`，`sklearn` 包，准备好数据集。这里使用的是MNIST手写数字数据集，训练集有60000张图片，测试集有10000张。

SVM原理

SVM的核心思想是通过优化一个凸二次规划问题，来在特征空间中找到最佳的超平面。它的基本模型是定义在特征空间上间隔最大的线性分类器。间隔是指点到超平面的间隔，间隔最大是指使得超平面和支持向量（距离分类超平面最近的那些样本点）之间的距离最大。

SVM能够执行线性或非线性分类、回归，异常值检测任务。对于非线性可分的情况，SVM可以通过核函数将输入特征映射到高维空间，使得原本线性不可分的数据在高维空间中变得线性可分。

CNN原理

CNN，即卷积神经网络，是一种经典的深度学习方法。

卷积神经网络由输入层、卷积和激活层、池化层、全连接和输出层组成。输入层接受原始的图像数据，卷积层将图像与卷积核进行卷积操作，并通过激活函数引入非线性。池化层通过减小特征图的大小来降低计算复杂度，全连接和输出层将特征转化为最终输出。其中，卷积和激活层、池化层都有多个。

SVM图片分类实现

第一步，引入 `tensorflow` 和 `sklearn`，引入MNIST数据集。

第二步，标记开始时间，对数据进行展平和标准化。这样，数据才能输入进SVM。

```
# 图像处理
x_train_flat = x_train.reshape(x_train.shape[0], -1)
x_test_flat = x_test.reshape(x_test.shape[0], -1)
scaler = StandardScaler()
x_train = scaler.fit_transform(x_train_flat)
x_test = scaler.fit_transform(x_test_flat)
```

第三步，进行训练。这里使用了高斯核函数。

```
# 训练
svm = svm.SVC(kernel='rbf', gamma='scale')
svm.fit(x_train, y_train)
```

第四步，进行测试，得出准确率和运行时间数据。

CNN图片分类实现

第一步，引入 `tensorflow`，引入MNIST数据集。

第二步，标记开始时间，调用 `ResNet50` 预训练模型，冻结模型原本的参数。

```
base_model = tf.keras.applications.ResNet50(weights='imagenet', include_top=False, input_shape=(224, 224, 3))
base_model.trainable = False
```

第三步，对模型进行微调。调用 `Sequential API`，在 `base_model` 上添加一个全局平均池化层转化输出形状，一个全连接层输出数字的十个分类（0-9）。最后，编译模型。

```
base_model.trainable = False
model = tf.keras.models.Sequential([
    base_model,
    tf.keras.layers.GlobalAveragePooling2D(),
    tf.keras.layers.Dense(units=10, activation='softmax')
])
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])
batch_size = 3000 # 每批次大小
```

第四步，训练模型。由于训练用时过长，将训练集缩小到12000张图片。同时，由于显存有限，将图片分批输入，每次3000张。

经过尝试，发现训练epoch=2时就可以实现90%以上的分类准确率。

```

batch_size = 3000 # 每批次大小
for i in range(0, 12000, batch_size):
    batch = x_train[i:i + batch_size]
    batch_y = y_train[i:i + batch_size]

    # 将批次数据调整形状为 (batch_size, 28, 28, 1)
    batch = tf.expand_dims(batch, axis=-1)
    # 调整批次数据大小为 224x224
    batch_resized = tf.image.resize(batch, size=[224, 224])

    # 如果需要将灰度图像转换为 RGB
    batch_rgb = tf.image.grayscale_to_rgb(batch_resized)

    print(batch_rgb.shape) # 输出每批次的形状
    print(i)

    model.fit(batch_rgb, batch_y, epochs=2)

```

第五步，进行测试。测试集缩小到3000张图片。

```

batch_test = x_test[0:3000]
batch_test_y = y_test[0:3000]

batch_test = tf.expand_dims(batch_test, axis=-1)

batch_test_resized = tf.image.resize(batch_test, size=[224, 224])

batch_test_rgb = tf.image.grayscale_to_rgb(batch_test_resized)

test_loss, test_accuracy = model.evaluate(batch_test_rgb, batch_test_y)

```

实验结果与对比

SVM:

```

E:\学习\Python3.7\python.exe E:\学习\Pycharm\GraphSplit\Classify.py
(60000, 28, 28) (60000,)
accuracy: 0.9656
runtime: 494.4515814781189

Process finished with exit code 0

```

CNN:

```
Test Accuracy: 0.9453333616256714  
runtime: 659.9122955799103
```

两种方法的分类准确率相近，但SVM的运行时间远小于CNN。